LeanCode

Firebase suite

BUILD

Get to market quickly and securely with products that can scale globally

Build AI-powered experiences into your apps and accelerate app development with managed infrastructure, powered by Google Cloud, so you can focus on what matters most.

RUN

Run your app with confidence and deliver the best experience for your users

Launch, monitor, and iterate with AI-assistive tools that help you optimize your app's quality and experience.

Build

Authentication

We build digital products.

LeanCode

Authentication

- Email & password / Email & link (Spark: 5 sign-in emails/day)
- Google / Apple / Facebook / Microsoft / GitHub / Twitter
- Phone number & SMS (real SMS: Blaze plan only)
- Anonymous user
- Custom auth system (your backend mints the token)

We build digital products.
LeanCode

```dart
try {
  await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
} on FirebaseAuthException catch (e) {
  showError(switch (e.code) {
    'invalid-credential' ||
    'user-not-found' ||
    'wrong-password' => 'Wrong email or password.',
    'too-many-requests' => 'Too many attempts, try again later.',
    'network-request-failed' => 'No network connection.',
    _ => 'Sign-in failed (${e.code}).',
  });
}
```

Cloud Firestore

We build digital products.

LeanCode

Cloud Firestore

- NoSQL document database
- Flexible & scalable - you just use it (free: 50K reads, 20K writes/day)
- Ordering & limiting
- Compound queries & indexing
- Security connected with Firebase Auth

We build digital products.

LeanCode

```dart
Future<void> addUser() async {
  try {
    final doc = await FirebaseFirestore.instance.collection('users').add({
      'full_name': fullName,
      'company': company,
      'age': age,
    });
    debugPrint('User added: ${doc.id}');
  } on FirebaseException catch (e) {
    debugPrint('Failed to add user: ${e.code}');
  }
}
```

Realtime Database

We build digital products.

LeanCode

Realtime Database

- Original Firebase DB
- Inspired Cloud Firestore (a separate database)
- Focused on low-latency synchronization
- Simple queries only

We build digital products.

LeanCode

```dart
Future<void> joinRoom(String uid) async {
  final me = FirebaseDatabase.instance.ref('rooms/flutter/online/$uid');
  await me.onDisconnect().remove();
  await me.set(true);

  FirebaseDatabase.instance
      .ref('rooms/flutter/online')
      .onValue
      .listen((event) => showOnlineCount(event.snapshot.children.length));
}
```

Realtime Database vs Cloud Firestore

	Realtime Database	Cloud Firestore
Data	One JSON tree	Collections of documents
Queries	One `orderBy`, filter on that key	Compound, OR, indexes
Offline	Android, iOS	Android, iOS, web
You pay for	GB stored and downloaded	Reads, writes, deletes
Presence	Built in: `onDisconnect()`	Not built in

Firestore is recommended. Can it do everything Realtime Database does?

Almost. `snapshots()` listeners sync in realtime too. What Firestore lacks is `onDisconnect()`, so presence (who is online) still needs Realtime Database. Firebase's presence guide for Firestore uses Realtime Database for this, plus a Cloud Function that copies the status into Firestore.

We build digital products.

LeanCode

Storage

We build digital products.

LeanCode

Storage

- File storage for user-generated content
- Robust operations (connectivity awareness)
- Integrated with Firebase Auth
- Scalability
- Requires the Blaze plan (no-cost quota still applies)

We build digital products.

LeanCode

```dart
Future<String?> uploadAvatar(String uid, Uint8List bytes) async {
  final ref = FirebaseStorage.instance.ref('users/$uid/avatar.png');
  try {
    await ref.putData(bytes, SettableMetadata(contentType: 'image/png'));
    return await ref.getDownloadURL();
  } on FirebaseException catch (e) {
    debugPrint('Upload failed: ${e.code}');
    return null;
  }
}
```

AI & Machine Learning

We build digital products.

LeanCode

AI & ML in Firebase today

- Gemini in your app: Firebase AI Logic (Spark: free tier)
- AI on the backend: Genkit (more later)
- On-device ML: ML Kit (Flutter: google_mlkit_* packages)
- Custom .tflite models (host them in Cloud Storage)
- Firebase ML: deprecated (shuts down 15 Jun 2027)

We build digital products.
LeanCode

```dart
Future<void> describePhoto(Uint8List photoBytes) async {
  final model = FirebaseAI.googleAI().generativeModel(
    model: 'gemini-3.8-flash',
  );

  final response = await model.generateContent([
    Content.multi([
      const TextPart('Describe this photo in one sentence.'),
      InlineDataPart('image/jpeg', photoBytes),
    ]),
  ]);
  showCaption(response.text ?? '');
}
```

We build digital products.

LeanCode

Hosting

Hosting

- SSL for free
- Static HTML (Flutter Web) or rewrite to Cloud Function
- Easy upload (through Firebase CLI)
- Cache / CDN included

We build digital products.

LeanCode

```
$ flutter build web
$ firebase init hosting
    public directory: build/web
    single-page app: yes
$ firebase deploy --only hosting
```

Cloud Functions

We build digital products.

LeanCode

Cloud Functions

- Integrated with Firebase triggers (Auth, Firestore, Storage)
- Serverless = no servers to manage (runtimes still expire)
- Deploy requires the Blaze plan (emulator works on Spark)
- Google Cloud version: Cloud Run functions (different CLI and SDK)

We build digital products.

LeanCode

```
const {onRequest} = require("firebase-functions/https");
const {initializeApp} = require("firebase-admin/app");
const {getFirestore} = require("firebase-admin/firestore");

initializeApp();

exports.addmessage = onRequest(async (req, res) => {
  const original = req.query.text;
  const writeResult = await getFirestore()
      .collection("messages")
      .add({original});
  res.json({result: `Message with ID: ${writeResult.id} added.`});
});
```

We build digital products.

LeanCode

```
const {onValueWritten} = require("firebase-functions/database");

exports.hello = onValueWritten("/hello", (event) => {
  return event.data.after.ref.set("world!");
});
```

We build digital products.

LeanCode

App Check

1. Your app interacts with the provider of your choice to obtain an attestation of the app or device's authenticity (or both, depending on the provider).
2. The attestation is sent to the App Check server, which verifies the validity of the attestation using parameters registered with the app, and returns to your app an App Check token with an expiration time. This token might retain some information about the attestation material it verified.
3. The App Check client SDK caches the token in your app, ready to be sent along with any requests your app makes to protected services.

App Check

iOS: App Attest, DeviceCheck

Android: Play Integrity (SafetyNet is shut down)

Web: reCAPTCHA Enterprise

Emulators and CI: debug provider

We build digital products.

LeanCode

```dart
await FirebaseAppCheck.instance.activate(
  providerAndroid: kDebugMode
      ? const AndroidDebugProvider()
      : const AndroidPlayIntegrityProvider(),
  providerApple: kDebugMode
      ? const AppleDebugProvider()
      : const AppleAppAttestWithDeviceCheckFallbackProvider(),
  providerWeb: ReCaptchaEnterpriseProvider('site-key'),
);
```

Shiny new features

SQL Connect

We build digital products.

LeanCode

Firebase SQL Connect

Build realtime, secure apps on PostgreSQL

An AI-native database solution for securely connecting web and mobile apps to a Cloud SQL for PostgreSQL database. Query, write and sync relational data in realtime.

Get started

View docs →



We build digital products.

LeanCode

Genkit

We build digital products.

LeanCode

Build agentic apps that do more than chat

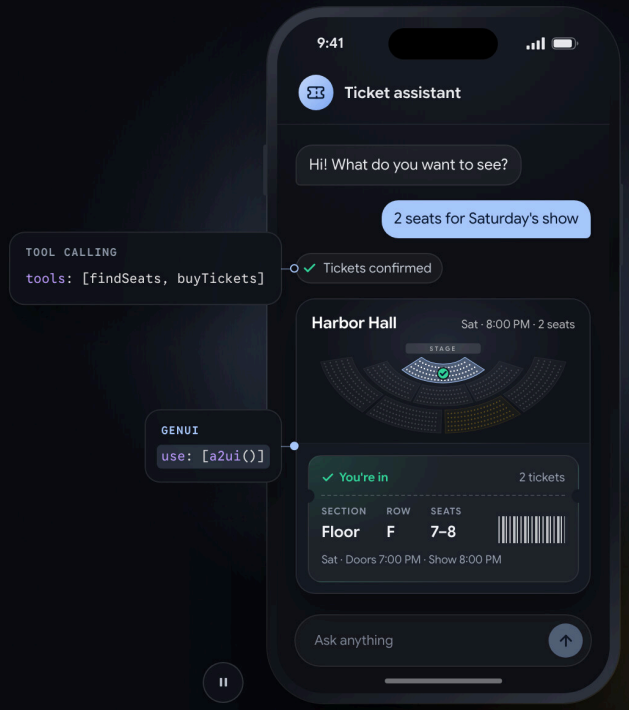
Genkit is an open-source AI SDK and toolkit for building full-stack agentic experiences for any platform. Available in TypeScript, Go, Dart, and Python

TypeScript | Go | Python | Dart | Agent skills

```
$ npm i genkit @genkit-ai/google-genai
```

Copy

Get started →



We build digital products.

LeanCode

Genkit for Dart

- 1.0 stable since Oct 2026 (pub.dev: genkit)
- Typed flows, tools, prompts, tracing
- Run in the Flutter app or on a Dart server
- Never embed private API keys in a published app

We build digital products.

LeanCode

Run

We build digital products.

LeanCode

Crashlytics

Crashlytics

- Crash reporting from Flutter
- Crash reporting from native code (iOS/Android) and Android NDK
- Deobfuscation handling
- Metrics synced with Analytics

We build digital products.

LeanCode

```dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();

  FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;

  PlatformDispatcher.instance.onError = (error, stack) {
    FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
    return true;
  };

  runApp(const MyApp());
}
```

We build digital products.

LeanCode

Performance Monitoring

We build digital products.

LeanCode

Performance Monitoring

- Startup times
- Custom code traces (no per-screen rendering on Flutter)
- HTTP requests
- Foreground / background activity
- Performance metrics

We build digital products.

LeanCode

Test Lab

We build digital products.

LeanCode

Test Lab

- Test on real devices and emulators
- Both Android and iOS
- Integrations with CLI / IDEs / CI systems
- integration_test package works with Test Lab
- Deprecated: shuts down 30 Sep 2027

We build digital products.

LeanCode

Test Lab + Flutter (Android)

- One-time setup: FlutterTestRunner (see integration_test README)
- Build two APKs: the app and the androidTest
- Upload both: Console or gcloud (type: instrumentation)
- Results per device: pass/fail, logs, video
- Spark: 10 virtual + 5 physical runs per day

We build digital products.

LeanCode

App Distribution

App Distribution

- Distribute artifacts
- Tester groups but only per-project
- No auto-provisioning
- No auto-update alerts
- Alternatives: TestFlight, Google Play (App Center is retired)

We build digital products.

LeanCode

Analytics

Analytics

- Quantity analytics
- Free, unlimited events (up to 500 event names)
- Events and user props
- Audience segmentation

```dart
await FirebaseAnalytics.instance.logEvent(
  name: 'test_event',
  parameters: {
    'string': 'string',
    'int': 42,
    'long': 12345678910,
    'double': 42.0,
    'bool': true.toString(),
  },
);
```

We build digital products.

LeanCode

Remote Config

We build digital products.

LeanCode

Remote Config

- Config server-side parameters for users
- App fetches config (min. every 12 h, or real-time)
- Connected to Analytics audiences

We build digital products.

LeanCode

```dart
Future<FirebaseRemoteConfig> setupRemoteConfig() async {
  final remoteConfig = FirebaseRemoteConfig.instance;
  await remoteConfig.setConfigSettings(
    RemoteConfigSettings(
      fetchTimeout: const Duration(seconds: 10),
      minimumFetchInterval: kDebugMode
          ? Duration.zero
          : const Duration(hours: 12),
    ),
  );
  await remoteConfig.setDefaults(const {
    'welcome': 'default welcome',
    'hello': 'default hello',
  });
  await remoteConfig.fetchAndActivate();
  remoteConfig.onConfigUpdated.listen((_) => remoteConfig.activate());
  return remoteConfig;
}
```

A/B Testing

A/B Testing

- Experiments with Remote Config
- Goals and targeting from Analytics
- Also tests notification and in-app message variants
- Allows “safe rollout” mechanisms (Remote Config rollouts)

A/B Testing inside Remote Config

- Create from Remote Config > Parameters (Create Experiment)
- Part of the template: publish to start or stop
- Same conditions as Remote Config (first match wins)
- Variant changes reach apps in real time (onConfigUpdated)

We build digital products.

LeanCode

Cloud Messaging (FCM)

We build digital products.

LeanCode

Cloud Messaging

- Send push notifications to Android / iOS / Web
- Use specific devices or pub/sub model
- Only a server sends (console, Admin SDK, HTTP v1)

We build digital products.

LeanCode

```dart
@pragma('vm:entry-point')
Future<void> _onBackgroundMessage(RemoteMessage message) async {
  debugPrint('Background message: ${message.messageId}');
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();

  FirebaseMessaging.onBackgroundMessage(_onBackgroundMessage);
  await FirebaseMessaging.instance.requestPermission();

  FirebaseMessaging.onMessage.listen((message) {
    final notification = message.notification;
    if (notification != null) {
      debugPrint('${notification.title}: ${notification.body}');
    }
  });
}
```

FCM: before the first push

- Android 13+: ask for permission (requestPermission() shows a dialog)
- Android emulator: Google Play image (others get no pushes)
- iOS: real device + APNs key (paid Apple Developer account)
- Web: VAPID key + service worker (small JS file that receives pushes)
- App open: no banner by default (see next slide)

We build digital products.

LeanCode

How a push reaches your Flutter app

App is...	Your code	What happens
Open on screen	`onMessage`	You get the data, but no banner pops up. Show it yourself.
In background or closed	`onBackgroundMessage`	Runs on the side, without your app's UI. Keep it short (~30 s).
User taps the push	`onMessageOpenedApp` `getInitialMessage()`	The app opens. You pick which screen.

Why `@pragma('vm:entry-point')`?

A release build deletes code that nothing in your app calls. Your app never calls the background handler: Android or iOS calls it from outside. The pragma says "don't delete this, it will be called from outside". That is also why it must be a top-level function, not a method of a class.

We build digital products.

LeanCode

In-App Messaging

We build digital products.

LeanCode

In-App Messaging

- Messages shown inside the app (fetched once a day, not instant)
- Use segmentation from Analytics
- Test now: “Test on Device” (needs the installation ID)
- Implement custom alerts in your code (setCustomDisplayEnabled)

We build digital products.

LeanCode

Google AdMob

We build digital products.

LeanCode

Google AdMob

- Just ads :)
- Can be connected to Firebase Analytics
- Flutter support

```
InterstitialAd.load(
  adUnitId: '<ad unit id>',
  request: AdRequest(),
  adLoadCallback: InterstitialAdLoadCallback(
    onAdLoaded: (InterstitialAd ad) {
      // Keep a reference to the ad so you can show it later.
      this._interstitialAd = ad;
    },
    onAdFailedToLoad: (LoadAdError error) {
      print('InterstitialAd failed to load: $error');
    },
  ));
```

We build digital products.

LeanCode

Emulator



We build digital products.

LeanCode

Firebase is a Backend as a Service (BaaS) for your app

We build digital products.

LeanCode

BaaS trade-off: you rent the backend

- Fast start, no servers to run
- But you also accept its lifecycle
- Shut down: Dynamic Links, Predictions
- Shutting down by 2027: ML, Extensions, Test Lab

We build digital products.

LeanCode